

Proyecto Final: Sistema de Detección y Conteo de Objetos para Aulas de Cómputo mediante Inferencia Local con YOLOv8 y TensorFlow.js

Autora: Lina Fernanda Rodriguez Balaguera, **Código:** 20252695007

Asignatura: Big Data - Módulo de Redes Convolucionales

Profesor: Gerardo Muñoz

Fecha: 16 de Noviembre de 2025

Resumen

Como aplicación práctica de los conceptos estudiados en el módulo de Redes Neuronales Convolucionales (CNNs), el presente proyecto aborda el diseño e implementación de un sistema de visión por computadora para la automatización del inventario en un salón de cómputo. El sistema debe ser capaz de identificar y contar seis clases de objetos predefinidos (CPU, Mesa, Mouse, Pantalla, Silla y Teclado) a partir de una única imagen, aplicando técnicas de detección de objetos, uno de los usos más avanzados y computacionalmente intensivos de las arquitecturas convolucionales.

1. Requisitos y Desafíos Clave

El verdadero desafío del proyecto no fue solo la detección en sí, sino las estrictas restricciones de implementación. El requisito clave era que la predicción del modelo debía correr 100% de forma local en el navegador del usuario, sin depender de servidores o APIs en la nube. Esta condición impuso un reto crítico: el tamaño del modelo fue un factor de evaluación principal, por lo que la eficiencia se volvió tan importante como la precisión.

Para cumplir con estos objetivos, el diseño del sistema se centró en balancear cuatro requisitos fundamentales que estaban en constante conflicto:

1. **Ejecución 100% Local:** La predicción debía correr en el navegador sin APIs externas. Esto nos obligó a descartar arquitecturas pesadas y seleccionar un formato web-ready (TensorFlow.js) capaz de ser descargado y ejecutado por el navegador del cliente.
2. **Precisión:** La aplicación debía detectar y contar correctamente los 6 objetos. Esto exigía una arquitectura de modelo (CNN) robusta y un dataset de alta calidad, que son factores que tienden a aumentar el tamaño del modelo.

3. Eficiencia de Tamaño: La evaluación del proyecto priorizaba un modelo "pequeño y eficiente". Este fue el principal desafío técnico, ya que creó un conflicto directo con el requisito de Precisión. Debíamos encontrar un modelo con el menor número de parámetros (MB) posible que aún fuera preciso.

4. Eficiencia de Velocidad (UX): La aplicación debía ser estable y no "congelarse" durante la predicción. Un modelo demasiado complejo, incluso si era pequeño, podría tardar varios segundos (o minutos, como se vio en la depuración inicial) en correr en el CPU/GPU del navegador, arruinando la experiencia de usuario.

Por lo tanto, el problema central fue encontrar una arquitectura CNN que, al ser entrenada, lograra el balance óptimo entre estos cuatro factores: máxima precisión con el mínimo tamaño de archivo y la mayor velocidad de ejecución local.

2. Solución Propuesta

Para abordar los desafíos de eficiencia, precisión y ejecución local, la solución implementada se basó en una arquitectura de vanguardia: YOLOv8-Nano. Este modelo fue entrenado con un dataset personalizado y, para cumplir con el requisito de inferencia 100% local, fue convertido al formato TensorFlow.js (TF.js). El sistema final se desplegó como una aplicación web estática (index.html) capaz de cargar el modelo y ejecutar la predicción directamente en el navegador del cliente.

2.1 La Eficiencia de YOLO

YOLO (acrónimo de "You Only Look Once") es una familia de arquitecturas CNN que revolucionó la detección de objetos en tiempo real. Su principal ventaja, y la razón de su elección para este proyecto, es su extraordinaria eficiencia. A diferencia de los detectores de dos etapas (como Faster R-CNN) que primero proponen regiones y luego las clasifican, YOLO es un detector de una sola etapa. Esto significa que trata la detección de objetos como un único problema de regresión: la red procesa la imagen completa una sola vez y predice simultáneamente todas las cajas delimitadoras (bounding boxes) y las probabilidades de clase en una única pasada.

Para este proyecto, se seleccionó específicamente la variante YOLOv8-Nano. Esta versión está diseñada para Edge AI y dispositivos con recursos limitados (como un navegador web), utilizando una arquitectura con un número drásticamente reducido de parámetros y capas convolucionales optimizadas. Esta elección resolvió directamente el conflicto planteado en los requisitos: el modelo yolov8n es lo suficientemente pequeño (solo 6.3 MB) para cumplir el requisito de eficiencia de tamaño, y su arquitectura es lo bastante rápida para ser ejecutada en segundos vía TensorFlow.js (usando WebGL), todo mientras mantenía una alta precisión (87.8% mAP en el dataset final).

3. Metodología

3.1 Gestión del Conjunto de Datos (Roboflow)

La calidad del modelo de detección depende directamente de la calidad del conjunto de datos. Para este proyecto, se utilizó la plataforma Roboflow como herramienta integral para la gestión, etiquetado, preprocesamiento y aumentación de las imágenes.

Adquisición de Datos: El conjunto de datos se construyó a partir de dos fuentes principales. En primera instancia, se aprovechó un *dataset* de imágenes aisladas de una actividad anterior. Para asegurar que el modelo pudiera manejar escenarios realistas del salón de cómputo, este conjunto se complementó con la grabación de nuevos videos que capturaban múltiples objetos (sillas, mesas, CPUs) en un mismo encuadre. A estos videos se les aplicó un proceso de extracción de fotogramas (*frames*) para generar imágenes de entrenamiento ricas en contexto.

Etiquetado y Clases: La totalidad del conjunto de imágenes (104) fue cargada en Roboflow, donde se realizó el etiquetado manual (anotación con cajas delimitadoras) para las 6 clases de objetos requeridas:

- CPU (0)
- Mesa (1)
- Mouse (2)
- Pantalla (3)
- Silla (4)
- Teclado (5)

Preprocesamiento y División del Dataset: Antes del entrenamiento, se definieron dos pasos de preprocesamiento cruciales:

1. Auto-Orient: Para corregir automáticamente la orientación de las imágenes basándose en sus metadatos EXIF.
2. Resize (Stretch to 640x640 px): Este es un paso mandatorio. La arquitectura YOLO requiere un tamaño de entrada estandarizado. Se eligió 640x640 por ser un balance óptimo entre precisión y velocidad de inferencia.

El *dataset* final, que tras la experimentación probó ser el más efectivo ("campeón"), consistió en 104 imágenes. Estas se dividieron aleatoriamente usando una proporción estándar de 80% (Train), 10% (Valid) y 10% (Test), resultando en 82 imágenes para entrenamiento, 11 para validación y 11 para la prueba final.

Aumentación de Datos (Augmentation): Para mejorar la robustez del modelo y prevenir el sobreajuste, se aplicaron aumentaciones ligeras al conjunto de entrenamiento. Estas transformaciones crean variaciones sintéticas de las imágenes:

- Flip: Volteo horizontal.
- Rotation: Rotaciones aleatorias entre -5° y $+5^\circ$.
- Brightness: Cambios aleatorios en el brillo entre -25% y $+25\%$.

3.2 Elección de la Arquitectura (YOLO vs. EfficientDet)

Para la selección del modelo, se evaluaron dos arquitecturas principales reconocidas por su eficiencia, comparando su viabilidad práctica y su alineación con los requisitos del proyecto.

La primera alternativa fue EfficientDet-Lite, una arquitectura nativa de TensorFlow. Esta opción se consideró inicialmente por su diseño optimizado para Edge AI y su tamaño de archivo extremadamente ligero, lo que la hacía teóricamente ideal para los requisitos de eficiencia. Sin embargo, esta vía se descartó en la fase de configuración. Su implementación en el entorno moderno de Google Colab presentó graves conflictos de dependencias (específicamente con protobuf y las versiones de TensorFlow 2.16+), lo que demostró ser un obstáculo insuperable para un desarrollo ágil y estable.

La alternativa B, y arquitectura seleccionada, fue YOLOv8-Nano. Se optó por esta arquitectura por ser el estándar actual de la industria, ofreciendo un entorno de entrenamiento mucho más rápido y robusto. La variante "nano" (yolov8n) resolvió perfectamente el conflicto central del proyecto: demostró ser práctica de implementar, precisa en la detección y, con un tamaño final de solo 6.3 MB, excepcionalmente eficiente, cumpliendo así con todos los requisitos de evaluación.

3.4 Entorno de Entrenamiento

Todo el proceso de entrenamiento del modelo se llevó a cabo en la plataforma Google Colab, seleccionada por su accesibilidad y su capacidad para proveer aceleración por hardware. Para manejar la alta demanda computacional del entrenamiento de redes convolucionales, se utilizó un runtime de GPU (NVIDIA T4). Este hardware fue crucial para reducir drásticamente los tiempos de entrenamiento (de horas a minutos) y permitir una rápida iteración experimental. El entrenamiento se gestionó programáticamente utilizando Python, y el modelo se implementó a través de la librería ultralytics, que proporciona la arquitectura oficial y las herramientas de entrenamiento para YOLOv8.

4. Experimentación y Resultados

Para alcanzar el equilibrio ideal entre la precisión del modelo predictivo y la eficiencia computacional requerida para su implementación, se implementó un proceso de entrenamiento riguroso e iterativo. Esta metodología se centró en la exploración y evaluación de una amplia gama de configuraciones.

Específicamente, se experimentó con múltiples combinaciones de *datasets* de entrenamiento, variando su tamaño, composición y calidad de las anotaciones. Paralelamente, se aplicaron y optimizaron diversas técnicas de aumentación de datos (*data augmentation*), tales como rotaciones, escalados, ajustes de brillo y contraste, y transformaciones de perspectiva.

La clave de este proceso de selección fue la comparación sistemática de las métricas de rendimiento. La métrica principal utilizada para la evaluación fue el mAP50 (Mean Average Precision al umbral de Intersección sobre Unión de 0.5), una medida estándar en tareas de detección de objetos que ofrece un balance sólido entre la identificación correcta de las instancias y la precisión de su localización. Mediante la comparación de los resultados de mAP50 de cada configuración probada, fue posible identificar y seleccionar el mejor modelo posible, aquel que no solo maximiza la precisión de las predicciones en el *dataset* de validación, sino que también mantiene una complejidad y tiempo de inferencia adecuados para su despliegue en el entorno operativo.

4.1 Experimento 1: Modelo Base (104 imágenes)

El primer modelo se entrenó con el dataset base de 104 imágenes, dividido en la proporción 80/10/10, y se le aplicaron aumentaciones ligeras (Flip, Rotation, Brightness). El entrenamiento se configuró para 100 épocas. Este modelo arrojó un resultado sobresaliente: un mAP50 general de 87.8%. Si bien esta precisión fue excelente para el tamaño del modelo, el análisis por clase reveló que la clase "Silla" era la más débil, con un mAP50 de solo 69.1%.

4.2 Experimento 2: Aumentación Agresiva (144 imágenes + Mosaic)

La hipótesis para el segundo experimento fue que la debilidad en la clase "Silla" se debía a la oclusión (objetos parcialmente ocultos). Para solucionarlo, se aumentó el dataset a 144 imágenes y se añadió la aumentación agresiva Mosaic. Sin embargo, esta configuración resultó contraproducente. El modelo obtuvo un mAP50 de 83.8% y el entrenamiento se detuvo prematuramente (EarlyStopping) en la época 50, al no observar mejoras. La conclusión fue que la tarea se volvió demasiado compleja para la capacidad del modelo nano, llevando a un underfitting (subajuste) y a un peor rendimiento.

4.3 Experimento 3: Datos "Ruidosos" (194 imágenes)

Para el tercer experimento, se intentó un enfoque de "datos difíciles", añadiendo 50 imágenes nuevas enfocadas únicamente en la clase "Silla", llevando el total a 194 imágenes. La hipótesis era que esta sobre-representación forzaría al modelo a aprender. Los resultados mostraron una ligera mejora sobre el Experimento 2, alcanzando un mAP50 de 85.5%, pero aún significativamente por debajo del modelo base. Se concluyó que los nuevos datos, al ser tan diferentes, actuaron como "ruido" y "contaminaron" el dataset, confundiendo al modelo y degradando su precisión general.

4.4 Modelo Final Seleccionado

Tras la experimentación, se concluyó irrefutablemente que el modelo del Experimento 1 fue el mejor modelo obtenido. Demostró que un dataset más pequeño pero de mayor calidad y con aumentaciones ligeras era superior a datasets más grandes pero ruidosos o con aumentaciones demasiado confusas.

Este modelo fue el seleccionado para la implementación final, ya que ofrecía la precisión más alta (mAP50 87.8%) con el tamaño óptimo (6.3 MB).

Las métricas finales de precisión para el modelo seleccionado se desglosan a continuación:

Clase	mAP50
Promedio General (all)	87.8%
CPU (0)	90.6%
Mesa (1)	95.3%
Mouse (2)	88.0%
Pantalla (3)	85.9%

Silla (4)	69.1%
Teclado (5)	97.7%

5. Implementación Web

La fase final del proyecto consistió en trasladar el modelo entrenado a un entorno web funcional, asegurando una inferencia local, rápida y estable.

5.1 Conversión del Modelo y Codificación de la Aplicación

El primer paso fue convertir el modelo ganador. El archivo `best.pt`, que es un checkpoint de PyTorch, no puede ser leído por un navegador. Utilizando la librería `ultralytics`, se ejecutó el comando de exportación para convertir el modelo al formato `TensorFlow.js`:

```
!yolo export model=.../best.pt format = tf.js
```

Este proceso generó la carpeta `best_web_model/`, que contiene el archivo `model.json` (que describe la arquitectura de la red) y un conjunto de archivos `.bin` (que contienen los pesos entrenados).

Posteriormente, se codificó el archivo `index.html`. Esta página actúa como el orquestador:

1. Importa la librería `TensorFlow.js` (`TF.js`) desde una CDN.
2. Carga el modelo (`model.json`) y los pesos en la memoria del navegador.
3. Un event listener en un `<input type="file">` espera la carga de una imagen.
4. La imagen se pre-procesa: se convierte en un tensor y se redimensiona a 640x640.
5. Se ejecuta la inferencia local (`model.executeAsync(tensor)`).
6. Los resultados (predicciones crudas) pasan a una función de post-procesamiento (`processOutput`) que aplica la lógica de NMS (Non-Max Suppression) y filtra por confianza.
7. Finalmente, las funciones `drawResults` y `displayCounts` dibujan las cajas en un `<canvas>` y actualizan el conteo en el HTML.

5.2 Ajustes de Rendimiento y Estabilidad en TensorFlow.js

La implementación inicial, aunque funcional, era inutilizable. El verdadero desafío fue la depuración en el navegador, que reveló cuatro *bugs* críticos que debieron ser solucionados:

1. Bug de Velocidad (Lentitud de +1 min):

- Problema: La inferencia inicial tardaba más de un minuto. El navegador estaba recurriendo al backend de CPU, que es ineficiente para los 3 millones de parámetros del modelo.
- Solución: Se forzó a TensorFlow.js a usar el backend de GPU del navegador (webgl) mediante el comando `await tf.setBackend('webgl')`. Esto redujo el tiempo de inferencia a un promedio de 3-5 segundos.

2. Bug de "Race Condition" (Condición de Carrera):

- Problema: La aplicación fallaba con un error `model is undefined`. Esto ocurría porque el usuario podía subir una imagen antes de que el modelo de 6.3 MB terminara de descargarse y cargarse en memoria.
- Solución: Se deshabilitó el botón `<input>` por defecto (`disabled`). Solo después de que la promesa de carga del modelo (`tf.loadGraphModel`) se resuelve exitosamente, el botón se activa (`imageUpload.disabled = false;`).

3. Bug de Detección (0 Objetos Detectados):

- Problema: El modelo, que tenía un 87.8% de mAP, no detectaba ningún objeto en la aplicación.
- Solución: El error estaba en el post-procesamiento. Se reescribió la función `processOutput` para manejar la lógica de salida de YOLOv8 (que no tiene un "objectness score" separado) y se corrigió un bug crítico donde el filtro de confianza (`score_threshold`) del NMS estaba fijo en 0.4, descartando todas las detecciones válidas pero menos seguras.

4. Bug de Estabilidad (UI Congelada):

- Problema: Durante la inferencia, la interfaz de usuario se "congelaba" (locked UI thread).
- Solución: Se descubrió que la función `nonMaxSuppression()` era síncrona. Se refactorizó el código para usar `async/await` y la función `tf.image.nonMaxSuppressionAsync()`, permitiendo que el NMS se ejecute sin bloquear el navegador.

5.3 Despliegue (GitHub Pages)

El sistema final se desplegó en GitHub Pages para permitir el acceso público. Durante el despliegue inicial, se encontraron errores 404, ya que GitHub Pages intentaba procesar el repositorio con Jekyll, lo cual fallaba. Para solucionar esto, se añadió un archivo vacío llamado

.nojekyll en la raíz del repositorio, forzando a GitHub a servir la carpeta Inventario/ como un sitio de archivos estáticos.

6. Conclusiones

Se cumplieron todos los objetivos del proyecto, entregando una aplicación web 100% local. La elección del modelo YOLOv8-Nano fue un acierto, logrando un balance ideal entre eficiencia y precisión. El modelo final pesa solo 6.3 MB y, gracias a la optimización del backend a webgl, realiza inferencias en el navegador en menos de 5 segundos.

La experimentación con los datos fue crucial. Se concluyó que la calidad y consistencia del dataset son más importantes que la cantidad. Nuestro mejor modelo (87.8% mAP) se logró con las 104 imágenes originales, que poseían una alta calidad de resolución. Los intentos posteriores de añadir más datos (llegando a 194 imágenes) resultaron ser contraproducentes, reduciendo la precisión. Se determinó que estas nuevas imágenes, además de ser menos variadas, no compartían la misma calidad de resolución que el conjunto original, actuando como "ruido" de baja fidelidad que degradó el rendimiento general del modelo nano.

Finalmente, el proyecto fue una demostración práctica de los conceptos fundamentales del módulo de Redes Convolucionales. La elección de la arquitectura no fue trivial: en lugar de arquitecturas pesadas (como VGG o ResNet) diseñadas para la máxima precisión, el requisito de eficiencia (6.3 MB) nos obligó a seleccionar una arquitectura CNN ligera (YOLOv8-Nano), optimizada para inferencia rápida. Más importante aún, el proceso de experimentación para encontrar el mejor modelo fue un ejercicio directo para encontrar el punto de convergencia óptimo: el entrenamiento de 100 épocas representó el balance perfecto donde la función de pérdida (loss) en el conjunto de validación alcanzó su mínimo, logrando la más alta precisión (mAP) sin caer en el sobreajuste (overfitting) que se observó en experimentos con entrenamientos más largos o datasets "ruidosos".